Data Structures Home Work Set 8

Graphs and Hashing

Register No:2025115044

Name: Dharshan Prabhu S.B

Q1. Simulate BFS and DFS traversals for the given graph.

Code:

```
#include <stdio.h>
#define V 6
int g[V][V],seen[V],q[V];
void dfs(int u){int v;seen[u]=1;printf("%d ",u);for(v=0;v<V;v++)if(g[u][v]&&!seen[v])dfs(v);}
int main(void){
 int i,j,start,front=0,rear=0;
 printf("Enter adjacency matrix (%d x %d):\n",V,V);
 for(i=0;i<V;i++)for(j=0;j<V;j++)scanf("%d",&g[i][j]);
 printf("Enter starting vertex (0-%d): ",V-1);scanf("%d",&start);
 printf("BFS: ");q[rear++]=start;seen[start]=1;
 while(front<rear){int u=q[front++];printf("%d ",u);for(i=0;i<V;i++)if(g[u][i]&&!seen[i]){seen[i]=1;q[rear++]=i;}}
 for(i=0;i<V;i++)seen[i]=0;printf("\nDFS: ");dfs(start);printf("\n");return 0;
}
```

Output:

```
Enter adjacency matrix (6 x 6):
0 1 1 0 0 0
1 0 0 1 1 0
1 0 0 0 0 1
0 1 0 0 0 0
0 1 0 0 0 0
0 0 1 0 0 0
Enter starting vertex (0-5): 0
BFS: 0 1 2 3 4 5
DFS: 0 1 3 4 2 5
```

Q2. Find the minimum spanning tree for the given graph using Prim's and Kruskal's algorithms.

Code:

```
#include <stdio.h>
#define V 4
#define E 5
#define INF 99999
struct Edge{int u,v,w;};
int parent[V];
int root(int x){while(parent[x]!=x){parent[x]=parent[parent[x]];x=parent[x];}return x;}
int main(void){
 int g[V][V],i,j,used[V]={0},key[V],par[V],total=0,min,u;struct Edge e[E],t;int k=0,chosen=0,ktotal=0;
 printf("Enter weighted adjacency matrix (0 means no edge):\n");for(i=0;i<V;i++)for(j=0;j<V;j++)scanf("%d",&g[i][j]);
 for(i=0;i<V;i++){key[i]=INF;par[i]=-1;}key[0]=0;
 printf("Prim MST edges:\n");
 for(i=0;i<V;i++){min=INF;u=-
1;for(j=0;j<V;j++)if(!used[j]&&key[j]<min){min=key[j];u=j;}if(u<0)break;used[u]=1;if(par[u]!=-1){printf("%d-
```

```
%d: %d\n",par[u],u,key[u]);total+=key[u];}
  for(j=0;j<V;j++)if(g[u][j]>0&&!used[j]&&g[u][j]<key[j]){key[j]=g[u][j];par[j]=u;}}
 printf("Prim total = %d\n",total);
 for(i=0;i<V;i++)for(j=i+1;j<V;j++)if(g[i][j]>0)e[k++]=(struct Edge){i,j,g[i][j]};
 for(i=0;i<k-1;i++)for(j=0;j<k-i-1;j++)if(e[j].w>e[j+1].w){t=e[j];e[j]=e[j+1];e[j+1]=t;}
 for(i=0;i<V;i++)parent[i]=i;ktotal=0;chosen=0;printf("Kruskal MST edges:\n");
 for(i=0;i<k&&chosen<V-1;i++){int a=root(e[i].u),b=root(e[i].v);if(a!=b){parent[a]=b;printf("%d-
%d: %d\n",e[i].u,e[i].v,e[i].w);ktotal+=e[i].w;chosen++;}}
 printf("Kruskal total = %d\n",ktotal);return 0;
}
```

Output:

```
Enter weighted adjacency matrix (0 means no edge):
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
Prim MST edges:
0-3: 5
3-2: 4
0-1: 10
Prim total = 19
Kruskal MST edges:
2-3: 4
0-3: 5
0-1: 10
Kruskal total = 19
```

Q3. Find the minimum spanning tree for the additional weighted graph shown.

The photo appears to contain a further weighted graph, but the labels and weights are not clear enough to transcribe reliably. This Kruskal program accepts its edges as input. Replace the sample edges with those from the notebook diagram.

Code:

```
#include <stdio.h>
#define V 5
#define E 7
struct Edge{int u,v,w;};
int parent[V];
int find(int x){if(parent[x]!=x)parent[x]=find(parent[x]);return parent[x];}
int main(void){
 struct Edge e[E],t;int i,j,total=0,count=0;
 printf("Enter %d edges as: start end weight (vertices 0 to %d)\n",E,V-1);
 for(i=0;i<E;i++)scanf("%d%d%d",&e[i].u,&e[i].v,&e[i].w);
 for(i=0;i<E-1;i++)for(j=0;j<E-i-1;j++)if(e[j].w>e[j+1].w){t=e[j];e[j]=e[j+1];e[j+1]=t;}
 for(i=0;i<V;i++)parent[i]=i;
 printf("MST edges:\n");
 for(i=0;i<E&&count<V-1;i++){int a=find(e[i].u),b=find(e[i].v);if(a!=b){parent[a]=b;printf("%d-
%d: %d\n",e[i].u,e[i].v,e[i].w);total+=e[i].w;count++;}}
 if(count==V-1)printf("Total weight = %d\n",total);else printf("The graph is disconnected; an MST does not exist\n");
 return 0;
}
```

Output:

```
Enter 7 edges as: start end weight (vertices 0 to 4)
0 1 2
0 2 6
1 2 3
1 3 8
2 3 5
2 4 7
3 4 4
MST edges:
0-1: 2
1-2: 3
3-4: 4
2-3: 5
Total weight = 14
```

Q4. Implement Dijkstra's algorithm for the given weighted graph.

Code:

```
#include <stdio.h>
#define V 5
#define INF 99999
int main(void){
 int g[V][V],d[V],done[V]={0},i,j,src,min,u;
 printf("Enter weighted adjacency matrix (0 means no edge):\n");for(i=0;i<V;i++)for(j=0;j<V;j++)scanf("%d",&g[i][j]);
 printf("Enter source vertex (0-%d): ",V-1);scanf("%d",&src);for(i=0;i<V;i++)d[i]=INF;d[src]=0;
 for(i=0;i<V;i++){min=INF;u=-1;for(j=0;j<V;j++)if(!done[j]&&d[j]<min){min=d[j];u=j;}if(u==-1)break;done[u]=1;
  for(j=0;j<V;j++)if(g[u][j]>0&&!done[j]&&d[u]+g[u][j]<d[j])d[j]=d[u]+g[u][j];}
 printf("Shortest distances from %d:\n",src);for(i=0;i<V;i++)if(d[i]==INF)printf("%d: unreachable\n",i);else
printf("%d: %d\n",i,d[i]);return 0;
}
```

Output:

```
Enter weighted adjacency matrix (0 means no edge):
0 10 0 5 0
0 0 1 2 0
0 0 0 0 4
0 3 9 0 2
7 0 6 0 0
Enter source vertex (0-4): 0
Shortest distances from 0:
0: 0
1: 8
2: 9
3: 5
4: 7
```

Q5. Find a topological ordering for the given directed acyclic graph (DAG).

Code:

```
#include <stdio.h>
#define V 6
int main(void){
 int g[V][V],indeg[V]={0},q[V],front=0,rear=0,i,j,count=0;
```

```
 printf("Enter adjacency matrix of the
DAG:\n");for(i=0;i<V;i++)for(j=0;j<V;j++){scanf("%d",&g[i][j]);if(g[i][j])indeg[j]++;}
 for(i=0;i<V;i++)if(indeg[i]==0)q[rear++]=i;
 printf("Topological order: ");while(front<rear){int u=q[front++];printf("%d ",u);count++;for(j=0;j<V;j++)if(g[u][j]&&--
indeg[j]==0)q[rear++]=j;}
 if(count<V)printf("\nGraph contains a cycle; topological ordering is not possible");printf("\n");return 0;
}
```

Output:

```
Enter adjacency matrix of the DAG:
0 1 1 0 0 0
0 0 0 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Topological order: 0 1 2 3 4 5
```

Q6. Insert 78, 93, 40, 17, 10 and 55 into a hash table of size 7 using linear probing.

The hash function used is h(key) = key mod 7.

Code:

```
#include <stdio.h>
#define SIZE 7
int main(void){int h[SIZE],keys[]={78,93,40,17,10,55},i,j,index;for(i=0;i<SIZE;i++)h[i]=-1;
 for(i=0;i<6;i++){index=keys[i]%SIZE;j=0;while(j<SIZE&&h[(index+j)%SIZE]!=-1)j++;if(j==SIZE)printf("Table
full: %d not inserted\n",keys[i]);else h[(index+j)%SIZE]=keys[i];}
 printf("Index  : ");for(i=0;i<SIZE;i++)printf("%d ",i);printf("\nKey    : ");for(i=0;i<SIZE;i++)if(h[i]==-1)printf("- ");else
printf("%d ",h[i]);printf("\n");return 0;}
```

Output:

```
Index  : 0 1 2 3 4 5 6
Key    :  - 78 93 17 10 40 55
```

Q7. Insert 22, 30 and 25 into a hash table of size 7 using linear probing.

Use the hash function h(key) = key mod 7.

Code:

```
#include <stdio.h>
#define SIZE 7
int main(void){int h[SIZE],keys[]={22,30,25},i,j,index;for(i=0;i<SIZE;i++)h[i]=-1;
 for(i=0;i<3;i++){index=keys[i]%SIZE;j=0;while(j<SIZE&&h[(index+j)%SIZE]!=-
1)j++;if(j<SIZE)h[(index+j)%SIZE]=keys[i];}
 printf("Index  : ");for(i=0;i<SIZE;i++)printf("%d ",i);printf("\nKey    : ");for(i=0;i<SIZE;i++)if(h[i]<0)printf("- ");else
printf("%d ",h[i]);printf("\n");return 0;}
```

Output:

```
Index  : 0 1 2 3 4 5 6
Key    :  - 22 30  - 25  -  -
```

Q8. Insert 27, 43, 69 and 72 into a hash table of size 9 using double hashing.

Code:

```
#include <stdio.h>
#define SIZE 9
int main(void){int h[SIZE],keys[]={27,43,69,72},i,j,index,step;for(i=0;i<SIZE;i++)h[i]=-1;
 for(i=0;i<4;i++){index=keys[i]%SIZE;step=7-(keys[i]%5);j=0;while(j<SIZE&&h[(index+j*step)%SIZE]!=-
1)j++;if(j<SIZE)h[(index+j*step)%SIZE]=keys[i];else printf("Could not insert %d\n",keys[i]);}
 printf("Index : ");for(i=0;i<SIZE;i++)printf("%d ",i);printf("\nKey   : ");for(i=0;i<SIZE;i++)if(h[i]<0)printf("- ");else
printf("%d ",h[i]);printf("\n");return 0;}
```

Output:

```
Index : 0 1 2 3 4 5 6 7 8
Key   : 27  -  -  -  - 72 69 43  -
```

Q9. Prove that with quadratic probing and a prime-sized table, a new element can be inserted when the table is at most half full.

Answer:

Let the table size be an odd prime number p, and use the quadratic probing sequence

```
h(k, i) = (h(k) + i^2) mod p,    i = 0, 1, 2, ..., (p - 1) / 2.
```

The values $i^2 \bmod p$ for $i = 0$ through $(p - 1)/2$ are distinct. If $i^2 \equiv j^2 \pmod p$, then $(i - j)(i + j) \equiv 0 \pmod p$. Since p is prime, $i \equiv j$ or $i \equiv -j \pmod p$. In the stated range, the only possibility is $i = j$. Therefore, this probe sequence checks $(p + 1)/2$ distinct table positions.

If the table is at most half full, at most $(p - 1)/2$ positions are occupied. Since the probe sequence checks $(p + 1)/2$ distinct positions, at least one checked position must be empty. Thus the new element can be inserted. This guarantee assumes the conventional i^2 probing sequence and that the table size is prime.

Q10. Implement a hash table using separate chaining.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 7
struct Node{int key;struct Node*next;};
void insert(struct Node*table[],int key){int i=key%SIZE;struct Node*p=malloc(sizeof(*p));p->key=key;p-
>next=table[i];table[i]=p;}
int search(struct Node*table[],int key){struct Node*p=table[key%SIZE];while(p){if(p->key==key)return 1;p=p-
>next;}return 0;}
int main(void){struct Node*table[SIZE]={NULL},*p;int keys[]={10,17,24,31,8},i,key;
 for(i=0;i<5;i++)insert(table,keys[i]);
 printf("Separate chaining table:\n");for(i=0;i<SIZE;i++){printf("%d:",i);for(p=table[i];p;p=p->next)printf(" %d ->",p-
>key);printf(" NULL\n");}
 printf("Enter key to search: ");scanf("%d",&key);if(search(table,key))printf("Key found\n");else printf("Key not
found\n");return 0;}
```

Output:

```
Separate chaining table:
0: NULL
1: 8 -> NULL
2: NULL
3: 31 -> 24 -> 17 -> 10 -> NULL
4: NULL
5: NULL
6: NULL
```

```
Enter key to search: 24
Key found
```